Studio Nx

JS 307 — 오디오 회귀 테스트 실행 명령과 자체 시험

2026-10-21

JavaScript 307 — 실행 명령과 자체 시험 (`run.mjs`, `selftest.mjs`)

항목	내용
파일	`studionx/tests/audio/run.mjs`, `studionx/tests/audio/selftest.mjs`, `studionx/tests/audio/README.md`
작업	S1-12 오디오 회귀 테스트 기반: 비교 실행, 골든 갱신, 테스트 도구 자체 시험 (PRD 00 §7.1)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 306 실행 도구, 다음 없음 (S1-12의 마지막 문서)

1. 하는 일

`run.mjs`가 소리를 건드린 뒤에 돌리는 명령 하나입니다. 모든 테스트 프로젝트를 렌더링해 골든과 비교하고, 하나라도 달라지면 종료 코드 1로 끝납니다. 의도한 변경이면 같은 명령에 `--update`와 이유를 붙여 골든을 새로 씁니다.

명령	동작
`node studionx/tests/audio/run.mjs`	모든 프로젝트를 비교합니다. 표로 결과와 지표를 보여 줍니다.
`... --only synth_basic,sampler_drums`	이름으로 골라서 비교합니다.
`... --update "변경 이유"`	골든을 새로 쓰고, 이전 골든과의 차이와 이유를 `golden/CHANGELOG.md`에 한 줄 남깁니다. 이유가 없으면 거부합니다.
`node studionx/tests/audio/selftest.mjs`	테스트 도구가 믿을 만한지 확인합니다 (62건).

실행 결과의 모양입니다.

```
브라우저: Chrome/155.0.8059.85
프로젝트            결과  최대 차이   RMS dBFS  LUFS 차이  골든 LUFS  시간
effects_chain     PASS  0.000000    -inf      0.000     -6.7       0.2s
synth_basic       FAIL  0.026516    -48.2     0.285     -16.2      0.2s
        - 샘플 최대 차이 2.65e-2 > 0.001
        - 차이 RMS -48.2 dBFS > -80 dBFS
        - LUFS 차이 0.285 LU > 0.05 LU
```

2. 언제 쓰나

상황	할 일
오디오 엔진, 플러그인, 라우팅 코드를 바꿈	`run.mjs`를 돌려 전부 통과하는지 봅니다.
실패함	`out/<이름>.wav`를 들어 보고, 의도한 변화인지 판단합니다.
의도한 변화였음	`--update "이유"`로 골든을 갱신하고 변경 기록과 함께 커밋합니다.
의도하지 않은 변화였음	코드를 고칩니다. 골든은 건드리지 않습니다.
테스트 도구 자체를 고침	`selftest.mjs`를 돌립니다.

3. 코드

3.1 실행 명령 (`run.mjs`)

```
function appendChangelog(rows, reason, chrome) {
  if (!fs.existsSync(CHANGELOG)) {
    fs.writeFileSync(CHANGELOG, [
      '# 골든 WAV 변경 기록',
      '',
      '의도적으로 소리를 바꾼 경우에만 골든을 새로 씁니다 (`node studionx/tests/audio/run.mjs --update "이유"`). 이유와 이전 골든과의 차이를 여기에 남깁니다.',
      '',
      '| 날짜 | 프로젝트 | 이유 | 이전과의 차이 (최대 차이, RMS dBFS, LUFS 차이) | 브라우저 |',
      '|---|---|---|---|---|',
      '',
    ].join('\n'), 'utf-8');
  }
  const date = new Date().toISOString().slice(0, 10);
  const text = rows.map(function (r) {
    const diff = r.cmp ? num(r.cmp.maxDiff, 6) + ', ' + num(r.cmp.rmsDiffDb, 1) + ', ' + num(r.cmp.lufsDiff, 3) : '새로 만듦';
    return '| ' + date + ' | `' + r.name + '` | ' + reason.replace(/\|/g, '/') + ' | ' + diff + ' | ' + chrome + ' |';
  }).join('\n') + '\n';
  fs.appendFileSync(CHANGELOG, text, 'utf-8');
}
```

```
async function main() {
  const onlyArg = argValue('--only');
  const only = onlyArg ? onlyArg.split(',') : null;
  const update = process.argv.indexOf('--update') >= 0;
  const reason = update ? (argValue('--update') || '').trim() : '';
  if (update && (!reason || reason.indexOf('--') === 0)) {
    console.error('--update 에는 변경 이유가 필요합니다. 예: --update "Reverb 감쇠 계산 수정"');
    process.exit(2);
  }
  const projects = loadProjects(only);
  if (!projects.length) { console.error('실행할 프로젝트가 없습니다 (--only 이름을 확인하세요)'); process.exit(2); }

  const r = await openRenderer();
  const results = [];
  try {
    for (const p of projects) {
      const t0 = Date.now();
      const out = await r.render(buildArgs(p));
      const ms = Date.now() - t0;
      const gp = goldenPath(p.name);
      const old = fs.existsSync(gp) ? readWav(gp) : null;
      if (update) {
        const cmp = old && old.frames === out.frames ? compareRenders(out.channels, old.channels, out.sampleRate, p.tolerance) : null;
        const bytes = writeFloatWav(gp, out.channels, out.sampleRate);
        results.push({ name: p.name, ms: ms, cmp: cmp, wrote: bytes, pass: true });
        continue;
```

```
      }
      if (!old) { results.push({ name: p.name, ms: ms, pass: false, note: '골든이 없습니다. --update "이유"로 만드세요' }); continue; }
      const cmp = compareRenders(out.channels, old.channels, out.sampleRate, p.tolerance);
      if (!cmp.pass) {
        fs.mkdirSync(OUT_DIR, { recursive: true });
        writeFloatWav(path.join(OUT_DIR, p.name + '.wav'), out.channels, out.sampleRate);
      }
      results.push({ name: p.name, ms: ms, cmp: cmp, pass: cmp.pass });
    }
  } finally {
    r.close();
  }

  if (update) {
    appendChangelog(results, reason, r.version);
    results.forEach(function (x) { console.log('골든 갱신 ' + x.name + ' (' + (x.wrote / 1048576).toFixed(2) + ' MB)' + (x.cmp ? '  이전과 최대 차이 ' + num(x.cmp.maxDiff, 6) : '  새로 만듦')); });
    console.log('변경 기록: ' + path.relative(process.cwd(), CHANGELOG));
    return;
  }

  console.log('브라우저: ' + r.version);
  console.log('프로젝트'.padEnd(18) + '결과  ' + '최대 차이'.padEnd(12) + 'RMS dBFS'.padEnd(10) + 'LUFS 차이'.padEnd(10) + '골든 LUFS  시간');
  results.forEach(function (x) {
    if (!x.cmp) { console.log(x.name.padEnd(18) + 'FAIL  ' + x.note); return; }
    console.log(x.name.padEnd(18) + (x.pass ? 'PASS  ' : 'FAIL  ') + num(x.cmp.maxDiff, 6).padEnd(12) + num(x.cmp.rmsDiffDb, 1).padEnd(10) + num(x.cmp.lufsDiff, 3).padEnd(10) + num(x.cmp.lufsGolden, 1).padEnd(11) + (x.ms / 1000).toFixed(1) + 's');
    x.cmp.failures.forEach(function (f) { console.log('        - ' + f); });
  });
  const failed = results.filter(function (x) { return !x.pass; });
  console.log('\n' + (results.length - failed.length) + '/' + results.length + ' 통과');
  if (failed.length) {
    console.log('실패한 렌더링은 studionx/tests/audio/out/ 에 저장했습니다. 의도한 변경이면 --update "이유"로 골든을 갱신하세요.');
    process.exit(1);
  }
}
```

4. 자체 시험 (`selftest.mjs`)

회귀 테스트가 거짓으로 통과하면 아무것도 지켜 주지 못하므로, 도구 자체를 세 가지로 확인합니다.

종류	확인하는 것	건수
지표	LUFS가 표준값과 맞는지, 문턱 동작, 차이 계산, 판정 규칙, WAV 왕복 (305)	21
골든	프로젝트마다 골든이 있는지, 통과하는지, 피크가 풀스케일 아래인지, 샘플레이트와 프레임 수가 같은지 (골든 파일 확인 1건과 프로젝트당 3건)	16
재현성	같은 프로젝트를 다시 렌더링해도 최대 차이가 1e-6 이하인지	1
민감도	소리를 조금 바꾼 프로젝트가 골든과 비교해 반드시 실패하는지	22
소리 없음	모든 클립이 빠진 렌더링이 "소리가 거의 없음"으로 실패하는지	1

종류	확인하는 것	건수
프로젝트 수	테스트 프로젝트가 3개에서 5개 사이인지	1

민감도 시험이 가장 중요합니다. 프로젝트를 복사해 한 군데만 일부러 바꾸고, 그 결과가 골든과 비교해 실패해야 통과입니다.

영역	바꾼 것 (모두 실패해야 함)
Synth	컷오프 +5%, 마스터 볼륨 -0.1dB, 노트 하나 반음 올림, LFO 깊이 0.35에서 0.30
Sampler	One-Shot에서 Classic으로, 스네어 Decay, 하이햇 벨로시티 하나, 베이스 Root
이펙트	Delay 시간 0.25에서 0.26, Compressor 임계값 1dB, Reverb 감쇠 0.1초, EQ 0.5dB, 이펙트 순서(Compressor와 Delay), Limiter 천장
믹서	팬 0.1, 뮤트 해제, 클립 gain 1dB, 클립 음정 반음 하나, 클립 시작 위치 0.02마디
템포	템포 변경 140에서 141 BPM, 변경 위치 0.1마디, Synth 컷오프 40Hz

5. 최신 문법으로 쓰면

```
// argValue 와 결과 표 출력을 최신 문법으로 쓴 모양
const argValue = (flag) => {
  const i = process.argv.indexOf(flag);
  return i >= 0 ? process.argv[i + 1] : null;
};

results.forEach(({ name, pass, cmp }) =>
  console.log(`${name.padEnd(18)}${pass ? 'PASS' : 'FAIL'}  ${cmp.maxDiff.toFixed(6)}`));

const changelogRows = rows.map((r) => `| ${date} | \`${r.name}\` | ${reason} |`).join('\n');
```

전통 문법 (실제 코드)	최신 문법
`function (x) { return ... }`	화살표 함수
`'	' + date + '
`x.cmp ? ... : ...`를 풀어 쓴 `if`	구조 분해 `({ name, pass, cmp })`

6. 왜 이렇게

선택	이유
골든 갱신에 이유를 필수로 합니다	PRD §7.1이 "의도적으로 소리를 바꾼 경우에만, 변경 이유를 기록하고" 갱신하라고 했습니다. 이유 없이 `--update`를 쓰면 실패한 테스트를 통과시키려고 골든을 덮어쓰는 일이 쉬워집니다. 명령이 이유를 요구하면 그 순간 한 번 생각하게 됩니다.
변경 기록에 이전 골든과의 차이와 브라우저 버전을 남깁니다	나중에 "이때 소리가 얼마나 달라졌고 어떤 Chrome이었는지"를 알 수 있습니다. 브라우저를 올리고 소리가 달라졌다면 이 기록이 원인을 좁혀 줍니다.
실패한 렌더링을 `out/`에 WAV로 저장합니다	숫자만으로는 무엇이 달라졌는지 모릅니다. 들어 보면 바로 알 수 있고, 저장소에는 올리지 않도록 `.gitignore`에 넣었습니다.
종료 코드를 실패 1, 사용 오류 2로 나눴습니다	CI나 스크립트에서 "소리가 달라졌다"와 "실행 자체가 잘못됐다"를 구분합니다.

선택	이유
민감도 시험을 둡니다	허용 오차가 너무 넓으면 실제 변화를 놓치는데, 그것은 통과하는 테스트로는 알 수 없습니다. 소리를 일부러 바꿔 보고 실패하는지 확인해야 허용 오차가 적절하다는 증거가 됩니다.
이펙트 순서 시험은 Compressor와 Delay를 바꿉니다	처음에는 Delay와 Reverb를 바꿨는데 결과가 같아서 시험이 실패했습니다. 둘 다 선형이고 시간에 따라 변하지 않는 필터라서 순서를 바꿔도 결과가 같은 것이 수학적으로 맞습니다. 비선형인 Compressor와 바꾸면 순서가 결과에 영향을 줍니다.
프로젝트의 Gain을 키워 Limiter가 일하게 했습니다	처음에는 소리가 천장(-1dB)보다 작아서 Limiter가 아무것도 하지 않았고, 천장을 -3으로 바꿔도 소리가 같았습니다. 회귀 테스트가 Limiter를 지켜 주려면 Limiter가 실제로 작동하는 소리를 넣어야 합니다.
마스터 볼륨을 내려 피크를 풀스케일 아래로 맞췄습니다	부동소수점 렌더링은 1.0을 넘어도 값이 유지되지만, 실제로 내보내면 잘리는 소리를 기준으로 삼으면 안 됩니다. 처음 만든 프로젝트 둘이 1.3에 가까웠습니다.

7. 시험

- 자체 시험 62건이 모두 통과합니다. 지표 21건, 골든 16건, 재현성 1건, 민감도 22건, 소리 없음 1건, 프로젝트 수 1건입니다.
- 실제 코드 변경으로 확인했습니다. `synth.js`의 최고 음량 계수 0.3을 0.31(약 +3%)로 바꾸고 `run.mjs`를 돌리자, Synth를 쓰는 `synth_basic`(LUFS 차이 0.285 LU)과 `full_mix_tempo`(최대 차이 6e-3)만 실패했고 나머지 세 프로젝트는 통과했습니다. 원래대로 되돌린 뒤에는 다시 5개 모두 통과했습니다. 한 곳의 변화가 그 소리를 쓰는 프로젝트에만 잡히는 것을 확인한 것입니다.
- 시험 중 발견한 일은 세 가지이고 모두 시험이나 프로젝트의 문제였습니다. 이펙트 순서 시험의 기대(Delay와 Reverb는 순서가 결과를 바꾼다)가 수학적으로 틀렸고, Limiter가 작동하지 않는 소리였으며, 무음을 붙인 LUFS 비교의 기대가 표준 알고리즘의 경계 블록 동작과 달랐습니다. 앱 코드의 오류는 없었습니다.